ChemoInteract

Complete Documentation — Part 2 of 3

11 Deep Learning Models, Mathematical Algorithms, Security & Testing

Author: Ronit **Version:** 0.1.0
Framework: PyTorch 2.0+ **Focus:** Models & Algorithms

Contents

1 11 Deep Learning Architectures

1.1 The Model Base Class Contract

```python
class Model(nn.Module, ABC):
    @abstractmethod
    def unpack(self, batch: DrugPairBatch):
        """Extract required feature tensors from batch."""
        ...
    def forward(self, *args) -> torch.FloatTensor:
        """Execute forward pass and return probabilities in [0, 1]."""
        ...
```

`unpack()` decouples the training pipeline from model-specific input structures.

1.2 Category 1: Fully-Connected (FC) Models

1.2.1 DeepSynergy (Preuer et al., 2018)

Concatenates cell line context vector and 256-bit Morgan fingerprints of both drugs:

$$\text{Input} = [\text{context} \,\|\, \text{left\_fp} \,\|\, \text{right\_fp}] \in \mathbb{R}^{C+2D}$$

Feeds through 4 linear layers with ReLU activation and Dropout (0.5) before a Sigmoid output layer.

1.2.2 DeepDDI (Ryu et al., 2018)

Takes concatenated drug fingerprints [left_fp ‖ right_fp]. Uses 9 dense layers with Batch Normalization after the first layer.

1.2.3 MatchMaker (Kuru et al., 2021)

Processes each drug with context using a **shared sub-network**:

1. $h_L = \text{SubNetwork}([\text{left\_fp} \,\|\, \text{context}])$
2. $h_R = \text{SubNetwork}([\text{right\_fp} \,\|\, \text{context}])$ (identical weights)
3. $\hat{y} = \text{Predictor}([h_L \,\|\, h_R])$

1.3 Category 2: Graph Neural Network (GNN) Models

1.3.1 DeepDDS (Wang et al., 2021)

Uses dual GCN branches for molecular graphs + an MLP branch for context vector. Aggregates graph embeddings with Max Readout and combines with context embedding.

1.3.2 DeepDrug (Cao et al., 2020)

Uses 4 GCN layers per drug graph → Global Max Readout → Concatenation → BatchNorm → Dropout → Linear → Sigmoid.

1.3.3 CASTER (Huang et al., 2020)

Input is the bitwise OR union of substructures. Uses an encoder-decoder architecture with sparse dictionary coding:

$$c = (D^T D + \lambda I)^{-1} D^T z$$

Trained with `CASTERSupervisedLoss`.

1.3.4 GCNBMP (Chen et al., 2020)

Integrates **Highway Networks** ($\mathbf{y} = T \odot H(\mathbf{x}) + (1 - T) \odot \mathbf{x}$), **Attention Pooling**, and **FFT-based Circular Correlation** ($a \star b = \mathcal{F}^{-1}(\overline{\mathcal{F}(a)} \odot \mathcal{F}(b))$).

1.3.5 EPGCN-DS (Sun et al., 2020)

Uses 2 GCN layers with Mean Readout. Combines drug representations via element-wise addition (Deep Sets principle).

1.3.6 MR-GNN (Xu et al., 2019)

Combines GCNs with two LSTMs: a Border LSTM tracking layer-wise feature evolution and a Middle LSTM tracking joint drug pair representations.

1.4 Category 3: Attention Models

1.4.1 MHCADDI (Deac et al., 2019)

Implements atom-to-atom cross-drug co-attention:

$$e_{ij} = \frac{(W_K h_i)^T (W_K h_j)}{\sqrt{d}}, \quad \alpha_{ij} = \text{segment\_softmax}_j(e_{ij}), \quad m_i = \sum_{j \in B} \alpha_{ij} W_V h_j$$

Processes drug pair graphs jointly.

1.4.2 SSI-DDI (Nyamabo et al., 2021)

Uses stacked GCN + LSTM blocks to model substructure-to-substructure interactions between drugs.

2 Algorithms and Mathematical Formulation

2.1 Graph Convolution Network (GCN) Message Passing

$$h_v^{(l+1)} = \sigma\left(W^{(l)} \cdot \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v) \cup \{v\}} h_u^{(l)}\right)$$

Aggregates 1-hop neighbor atom features per layer.

2.2 Circular Correlation via FFT

$$(a \star b)[k] = \sum_{j=0}^{n-1} a[j] \cdot b[(j+k) \bmod n] = \mathcal{F}^{-1}\left(\overline{\mathcal{F}(a)} \odot \mathcal{F}(b)\right)$$

Reduces correlation complexity from $O(n^2)$ to $O(n \log n)$.

2.3 Adam Optimizer

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

2.4 Binary Cross-Entropy Loss

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)]$$

3 Security, Error Handling, and Troubleshooting

Common Error	Cause & Solution
`AssertionError`	Invalid dataset name. Fix: use exact names (`drugcombdb`, `drugcomb`, `twosides`, `drugbankddi`).
`CUDA out of memory`	Mini-batch size too large for VRAM. Fix: reduce `batch_size` from 5120 to 512 or 256.
`AttributeError: PackedGraph`	Missing GPU transfer method. Fix: import `ChemoInteract.compat` at startup.
`KeyError: node_feature`	TorchDrug version feature naming mismatch. Fix: `compat.py` resolves key mapping automatically.

4 Deployment and Testing

4.1 Local Installation Test Suite

```
# Clone and install in editable mode
git clone https://github.com/your-username/ChemoInteract.git
cd ChemoInteract/ChemoInteract_Main
pip install -e .

# Run pytest unit tests
python -m pytest tests/ -v
```

4.2 Saving and Loading Models

```
results = pipeline(dataset="drugcombdb", model="deepsynergy", epochs
    =100)
results.save("./output_dir/")  # Saves model.pkl + results.json

# Loading model
import torch
model = torch.load("./output_dir/model.pkl")
model.eval()
```